Veronica Figueroa

CS 1851: Machine Learning for Biology and Health

Homework 3 Final Reflection

There were two datasets used in this homework. The first was the heart disease dataset that lends itself to a classification problem, predicting the presence or absence of heart disease. It included 13 features that encompassed a variety of demographic and biological information about a patient, including age, sex, cholesterol levels, and more. I did not use PCA for this dataset, so all features were included. Furthermore, there were no missing values.

The second dataset was the biological aging dataset, which in turn lends itself to a regression problem, predicting biological age. It included 485,516 features, everything from tissue type and gender information to thousands of genomic markers. This dataset did include missing values, both numeric and categorical; to handle this, I used mean imputation for the numeric columns and most frequent imputation for the string categorical ones. To concatenate everything together, after imputation, I used one-hot encoding to convert the categorical data into a numerical format. Because this was an extremely high-dimensional dataset, I computed the kernel experiments both with and without PCA; when performing the former, I used 20 components.

For the heart disease classification kernel experiments, I tested three different kernels (linear, polynomial, RBF), C values (0.1, 1, 10), degrees (2, 3, 4), and two different gamma values (scale, auto). After looping through all possible combinations, I ordered the result table by descending recall metrics. I chose recall as my initial judgement metric because it is used to capture all possible positives. For disease diagnosis, it is better to misclassify something as a false positive and have it later proven incorrect through additional testing rather than a false negative and let a true disease go unnoticed.

Table 1. Classification Kernel Experiment Metrics

Kernel	**C**	**Degree**	**Gamma**	**Accuracy**	**Precision**	**Recall**	**F1**
Poly	1.0	3.0	NaN	0.907317	0.855932	0.980583	0.914027
Poly	0.1	4.0	NaN	0.712195	0.641026	0.970874	0.914027
RBF	10.0	NaN	Auto	0.975610	0.980392	0.970874	0.975610
RBF	10.0	NaN	Scale	0.975610	0.980392	0.970874	0.975610

While the first row in the list represents the kernel experiment with the best recall score, the overall best model is the 3rd row (same values as the 4th row, as well). With only a marginally lower recall and much higher accuracy, precision, and f1 metrics, the **RBF kernel** with C=10.0 and **gamma = auto** (or **scale**, since there is no difference). This kernel is able to capture the nonlinear interactions between features, which seems to be helpful here; however, all top RBF and polynomial models performed quite well (and better than the linear) The confusion matrix and ROC and precision-recall curves for this model are shown below:

Table 2. Heart Disease Classification Confusion Matrix

	Actual Positive	**Actual Negative**
Predicted Positive	TP = 100	FP = 2
Predicted Negative	FN = 3	TN = 100

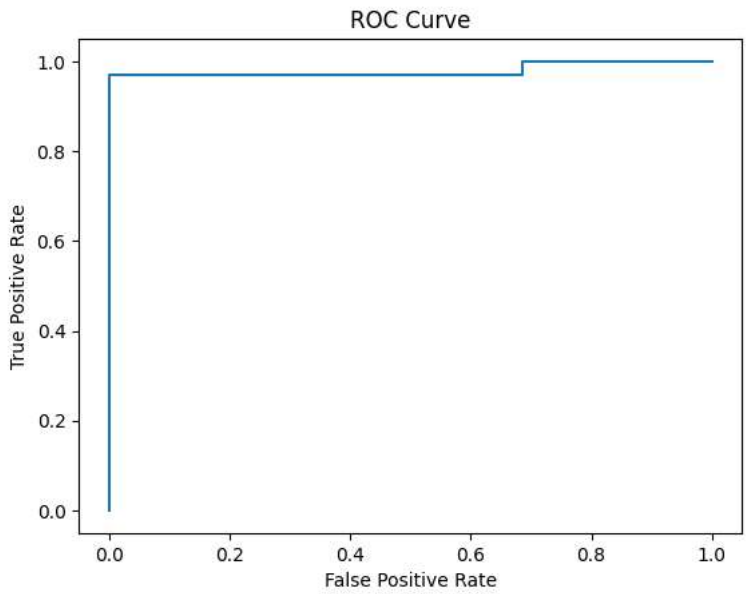

Figure 2. Heart Disease Classification ROC Curve

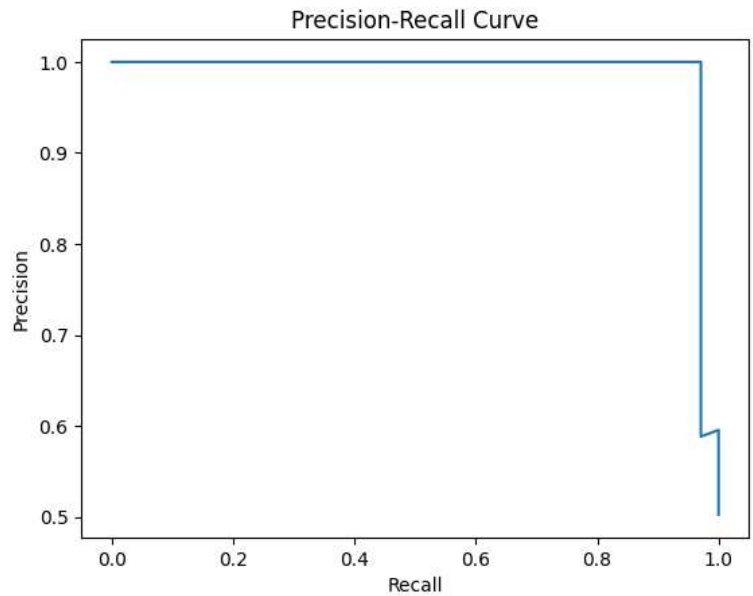

Figure 1. Heart Disease Classification Precision-Recall Curve

When examining hyperparameter sensitive for this dataset, the model performance seems most sensitive to C per kernel, which makes sense since the other hyperparameters are specific per kernel. Seen in the graphs in the Jupyter notebook, the linear kernels are the least sensitive and the RBF benefit the most from C tuning. Surprisingly, recall is affected very little by whether gamma is auto or scale (graphs produced in Jupyter notebook).

For the biological age regression kernel experiments, I tested the same types of kernels, C values, degrees, and gammas; however, I also computed one set with PCA (number of components = 20) and one without. The metrics are shown below:

Table 3. Regression Kernel Experiment Metrics WITHOUT PCA

Kernel	**C**	**Degree**	**Gamma**	**MAE**	**RMSE**	**R2**
RBF	1.0	NaN	Auto	4.198859	5.694251	0.001843
RBF	1.0	NaN	Scale	4.198860	5.69425	0.001843
Poly	1.0	4.0	NaN	4.114197	5.695620	0.001363
Poly	1.0	3.0	NaN	4.114197	5.699521	-0.000005

Table 4. Regression Kernel Experiment Metrics WITH PCA (num components = 20)

Kernel	**C**	**Degree**	**Gamma**	**MAE**	**RMSE**	**R2**
Poly	1.0	4.0	NaN	4.132680	5.689303	0.003577
RBF	1.0	NaN	Auto	4.237500	5.700384	-0.000308
Poly	1.0	3.0	NaN	4.089915	5.709999	-0.003685
Poly	0.1	3.0	NaN	3.936274	5.717326	-0.006263

I decided to rank results by R^2 since it measures the how much of the target variable variance the model actually explains. All models performed quite poorly, with the best models in both PCA and without PCA performing barely better than predicting the mean, even with the top PCA kernel experiment doubling the R^2 value. I also tested PCA with 5, 10, and 15 components – all results were worse than the 20 components. It is likely that the results are poor because this dataset is extremely high-dimensional, with many more features than samples. The model is probably making spurious correlations, learning trivial patterns that don't actually make sense, so PCA is not actually helping the way it should.

Similarly to the classification task, there is very little perturbance for linear models when C is tuned. RBF also seems to benefit the most from increased tuning. Furthermore, again similar to the classification task, there is very little difference between the auto and scale gamma hyperparameter, which is interesting, and leads me to wonder in what scenarios the choice does make a big difference (graphs produced in Jupyter notebook).